

LLMsVerifier

LLMsVerifier

सत्यापन करें। निगरानी करें। अनुकूलन करें।

सारांश

LLMsVerifier एक एंटरप्राइज़-ग्रेड प्लेटफ़ॉर्म है जो कई प्रदाताओं के लार्ज लैंग्वेज मॉडल्स (LLMs) का सत्यापन, निगरानी और अनुकूलन करता है। यह एक अनिवार्य “क्या तुम मेरा कोड देख रहे हो?” सत्यापन परीक्षण पर आधारित है, ताकि केवल वे मॉडल ही उपयोग के लिए चिह्नित या निर्यात किए जाएँ जो वास्तव में काम करते हों।

संक्षिप्त विवरण

Go एक ऐसा प्लेटफ़ॉर्म है जो कई प्रदाताओं के LLMs का सत्यापन, बेंचमार्किंग, निगरानी और अनुकूलन करता है। हर मॉडल को उपयोग से पहले एक अनिवार्य कोड-दृश्यता परीक्षण पास करना होता है; इसके बाद यह विलंबता, स्ट्रीमिंग, फ़ंक्शन-कॉलिंग, विज़न और एम्बेडिंग जाँच करता है, और केवल सत्यापित कॉन्फ़िगरेशन को AI CLI टूल्स के लिए निर्यात करता है।

विस्तृत विवरण

LLMsVerifier एक व्यापक प्लेटफ़ॉर्म है जो कई प्रदाताओं के LLM प्रदर्शन का सत्यापन, निगरानी और अनुकूलन करता है। इसका मूल सिद्धांत है अनिवार्य सत्यापन—और इसमें कोई समझौता नहीं किया जाता: किसी भी मॉडल को उपयोग के लिए चिह्नित करने या निर्यातित कॉन्फ़िगरेशन में शामिल करने से पहले, उसे एक “क्या तुम मेरा कोड देख रहे हो?” परीक्षण पास करना अनिवार्य है। यह परीक्षण प्रदाता को वास्तविक HTTP कॉल करता है और प्रतिक्रिया का विश्लेषण करता है कि मॉडल ने वास्तविक समझ दिखाई है या केवल विश्वसनीय दिखने वाला प्रतिध्वनि दिया है। जो मॉडल आपके इनपुट को स्पष्ट रूप से नहीं देख और समझ सकता, उसे कभी “उपयोगी” का टैग नहीं मिलता। इस जाँच के बाद, वेरिफ़ायर इंजन क्षमता परीक्षणों की एक पूरी शृंखला चलाता है—उपलब्धता, प्रतिक्रिया क्षमता, विलंबता, स्ट्रीमिंग, फ़ंक्शन कॉलिंग, विज़न, embeddings—और रिपोर्टर इंजन इन परिणामों को मार्कडाउन और JSON रिपोर्ट्स में बदलता है, जिन पर आप कार्रवाई कर सकते हैं।

यह प्रणाली मॉड्यूलर और इवेंट-ड्रिवन है, जो वेरिफायर इंजन, रिपोर्टर इंजन और कॉन्फिगरेशन मैनेजर के मूल पर CLI, TUI, वेब और REST API इंटरफ़ेस प्रदान करती है। यह सत्यापन तक सीमित नहीं है। उन्नत परतें LLM-संचालित कार्य विघटन के लिए सुपरवाइज़र/वर्कर पैटर्न जोड़ती हैं, स्लाइडिंग-विंडो और LLM-सारांशित संदर्भ प्रबंधन ताकि लंबे सत्र अचानक बाधित न हों, क्लाउड-समर्थित चेकपॉइंटिंग, और एक फ़ेलओवर प्रणाली जिसमें सर्किट ब्रेकर और विलंबता-आधारित रूटिंग शामिल है। आसपास का इंफ्रास्ट्रक्चर प्रोडक्शन-तैयार है: एक पब/सब इवेंट बस, क्रॉन शेड्यूलिंग, मूल्य/सीमा पहचान, RAG के लिए vector डेटाबेस, और एक निर्यात प्रणाली। एक विशिष्ट ब्रांडिंग कन्वेंशन हर जनरेटेड प्रदाता/मॉडल के नाम के अंत में (llmsvd) प्रत्यय जोड़ता है, ताकि सत्यापित आउटपुट एक नज़र में पहचाना जा सके और कभी भी असत्यापित मॉडल से भ्रमित न हो—और केवल सत्यापित मॉडल ही AI CLI टूल्स जैसे OpenCode, Crush और Claude Code के लिए निर्यातित कॉन्फिगरेशन में लिखे जाते हैं। यह प्रोडक्शन में टीमों को वास्तव में आवश्यक ऑपरेशनल टूलिंग के साथ आता है: Docker/ Kubernetes/Helm डिप्लॉयमेंट, Prometheus/Grafana मॉनिटरिंग, LDAP/ SSO, और SQLCipher-एन्क्रिप्टेड स्टोरेज।

हमने इसे क्यों बनाया

क्योंकि केवल कॉन्फिगरेशन-आधारित जाँच विश्वसनीय नहीं है—एक API कुंजी समाप्त हो सकती है, एक मॉडल अप्रचलित हो सकता है, और कॉन्फिग फ़ाइल आपको वास्तविक विलंबता, वास्तविक त्रुटियों या यह नहीं बताती कि मॉडल वास्तव में आपका इनपुट देख और समझ सकता है या नहीं। LLMsVerifier “यह कॉन्फिगरेशन में है, इसलिए यह काम करना चाहिए” की जगह प्रमाण रखता है: केवल वे मॉडल ही उपयोगी चिह्नित और निर्यात किए जाते हैं जो वास्तव में सही प्रतिक्रिया देते हैं।

सामग्री

यह क्यों एक गेम-चेंजर है

यह LLM के बेड़ों को विश्वसनीय बनाता है — एक ऐसा शब्द जो उस क्षेत्र में शायद ही कभी अर्जित होता है जहाँ कॉन्फिगरेशन चुपके से झूठ बोलते हैं। यह उम्मीद करने के बजाय कि कोई कॉन्फिगर किया गया मॉडल काम करेगा, टीमों को एक लागू, परीक्षण योग्य गारंटी मिलती है कि हर सक्रिय मॉडल ने वास्तविक सत्यापन पास किया है, जिसमें निगरानी, फ़ेलओवर और केवल-सत्यापित निर्यात शामिल हैं जो प्रमाण से उत्पादन तक के चक्र को पूरा करते हैं। Helix पारिस्थितिकी तंत्र के भीतर यह LLM मॉडल, प्रदाता और सत्यापन मेटाडेटा के लिए एकमात्र सत्य का स्रोत बन जाता है: अन्य सेवाएँ (जिनमें HelixTranslate भी शामिल है) इसी के आधार पर रूटिंग करती हैं, जिससे पूरी प्लेटफ़ॉर्म को एक ईमानदार उत्तर मिलता है — “वर्तमान में कौन से मॉडल वास्तव में काम कर रहे हैं?” — बजाय इसके कि हर टीम अपनी-अपनी आशावान अटकलें लगाती रहे।

क्या नवीन है

- अनिवार्य “क्या तुम्हें मेरा कोड दिख रहा है?” सत्यापन — एक वास्तविक, HTTP-समर्थित समझ की जाँच जिसे हर मॉडल को उपयोग से पहले पास करना अनिवार्य है; यह उत्पाद की विशिष्ट पहचान है और यही कारण है कि कोई भी अप्रमाणित चीज़ छूट नहीं पाती।

- **केवल-सत्यापित कॉन्फ़िगरेशन निर्यात** — AI CLI टूल्स के लिए उत्पन्न कॉन्फ़िगरेशन में केवल वे मॉडल शामिल होते हैं जो सत्यापन पास कर चुके हैं, ताकि आप जो कॉन्फ़िगरेशन भेजें वह चुपचाप किसी टूटे हुए मॉडल को फिर से शामिल न कर सके।
- **(LLmsvd) ब्रांडिंग-सफ़िक्स प्रणाली** — हर उत्पन्न प्रदाता/मॉडल के साथ एक पता लगाने योग्य सफ़िक्स जुड़ा होता है, जिससे सत्यापित उत्पत्ति हर उस जगह दिखाई देती है जहाँ आउटपुट जाता है।
- **कई CLI एजेंटों और प्रदाताओं में क्षमता पहचान** — यह स्ट्रीमिंग प्रकारों (SSE, WebSocket, JSONL, EventStream), संपीड़न और कैशिंग व्यवहार को फ़िगरप्रिंट करता है बजाय इसके कि उन्हें मान लिया जाए।
- **लचीला फ़ेलओवर** — सर्किट ब्रेकर, विलंबता-आधारित रूटिंग जो पहले-टोकन के समय सीमा पार करने पर पुनर्निर्देशित करती है, स्वास्थ्य जाँच और भारित ट्रैफ़िक विभाजन बड़े को तब भी प्रतिक्रियाशील बनाए रखते हैं जब अलग-अलग प्रदाता अस्थिर हो जाते हैं।
- **दीर्घकालिक स्वायत्तता** — सुपरवाइज़र/वर्कर डीकंपोज़िशन पैटर्न के साथ-साथ चेकपॉइंटिंग और मेमोरी एकीकरण लंबे सत्रों को बनाए रखते हैं जो अन्यथा संदर्भ से बाहर हो जाते।
- **भू-संदर्भित संदर्भ संवर्द्धन के लिए RAG / vector-DB एकीकरण**।

सबसे बड़ी तकनीकी चुनौतियाँ और उनके समाधान

- **साबित करना कि कोई मॉडल वास्तव में काम करता है, न कि केवल यह कि वह कॉन्फ़िगर किया गया है।** यही मुख्य उद्देश्य है और सबसे कठिन हिस्सा। इसका समाधान अनिवार्य कोड-दृश्यता परीक्षण से किया गया, जो वास्तविक API कॉल करता है और प्रतिक्रियाओं का विश्लेषण करके सकारात्मक समझ सुनिश्चित करता है, साथ ही क्षमता परीक्षणों के व्यापक सूट द्वारा समर्थित — और फिर जो पास नहीं हुआ उसे निर्यात करने से इनकार करके, ताकि कॉन्फ़िगरेशन नहीं बल्कि प्रमाण ही उत्पादन को नियंत्रित करे।
- **कई अस्थिर तृतीय-पक्ष प्रदाताओं में विश्वसनीयता।** इसका समाधान फ़ेलओवर ऑर्किस्ट्रेटर से किया गया जो प्रदाता अस्थिरता को सामान्य स्थिति मानता है: सर्किट ब्रेकर N विफलताओं के बाद M सेकंड में प्रदाता को डिग्रेडेड चिह्नित कर देते हैं, विलंबता-आधारित रूटिंग धीमे एंडपॉइंट से दूर ले जाती है, आवधिक स्वास्थ्य जाँच पुनर्प्राप्ति की जाँच करती है, और भारित रूटिंग लागत-प्रभावी और प्रीमियम मॉडलों के बीच संतुलन बनाती है।
- **बहुत लंबे, स्वायत्त सत्रों को बनाए रखना।** इसका समाधान सुपरवाइज़र/वर्कर डीकंपोज़िशन पैटर्न से किया गया जो बड़े कार्य को प्रबंधनीय टुकड़ों में विभाजित करता है, आवधिक चेकपॉइंटिंग क्लाउड स्टोरेज में प्रगति को सुरक्षित रखती है ताकि व्यवधान आने पर भी काम न रुके, और स्तरित संदर्भ प्रबंधन (स्लाइडिंग विंडो + LLM संक्षेपण + RAG) मॉडल को टोकन में डूबने से बचाते हुए थ्रेड बनाए रखता है।
- **प्रदाता विस्तार।** इसका समाधान एक सामान्य इंटरफ़ेस के पीछे कई प्रति-प्रदाता Go एडेप्टर छिपाकर किया गया, जहाँ वास्तविक एंडपॉइंट केंद्रीय रूप से सूचीबद्ध होते हैं — ताकि किसी प्रदाता को जोड़ना एक सीमित बदलाव हो, न कि पूरे कोडबेस में तरंग प्रभाव।

सामग्री

तकनीकी स्टैक

- **Go** — कोर प्लेटफॉर्म भाषा के रूप में चुना गया इसके समवर्ती (कंकरेंसी) गुण के कारण; यह एक बहु-थ्रेडेड वेरिफायर इंजन को संचालित करता है जो कई मॉडलों की समानांतर जांच कर सकता है, साथ ही आसपास की सेवाओं को भी।
- **Gin** — **REST API** सर्वर के रूप में चुना गया, जिसमें **JWT** प्रमाणीकरण, दर सीमित करने की सुविधा, और **WebSocket/SSE** एंडपॉइंट्स शामिल हैं।
- **SQLite + SQLCipher** — एम्बेडेड स्टोरेज के लिए चुना गया डेटाबेस-स्तरीय एन्क्रिप्शन के साथ, क्योंकि वेरिफिकेशन डेटा (कंजियाँ, परिणाम) संवेदनशील होता है और इसे डिफॉल्ट रूप से आराम की स्थिति में एन्क्रिप्टेड रखना चाहिए।
- **Redis** — कैशिंग परत के रूप में चुना गया ताकि गर्म वेरिफिकेशन और मेटाडेटा लुकअप तेज़ रहें।
- **RabbitMQ + Kafka** — इवेंट-ड्रिवन आर्किटेक्चर को शक्ति प्रदान करने के लिए चुना गया: मैसेजिंग और स्ट्रीमिंग जो प्लेटफॉर्म भर में उत्पादकों को उपभोक्ताओं से अलग करती है।
- **gRPC + Protocol Buffers** — घटकों के बीच मजबूत टाइपिंग वाले अंतर-सेवा संचार और इवेंट ट्रांसपोर्ट के लिए चुना गया।
- **QUIC / HTTP-3 (quic-go)** — आधुनिक ट्रांसपोर्ट समर्थन के लिए चुना गया (रिपॉजिटरी दस्तावेज़ **HTTP/3** प्रदाता उपलब्धता को सीमित बताते हैं — यह एक क्षमता है, सार्वभौमिक दावा नहीं)।
- **JWT + LDAP/NTLM** — एंटरप्राइज़ प्रमाणीकरण के लिए चुना गया ताकि प्लेटफॉर्म मौजूदा कॉर्पोरेट पहचान (SSO/SAML/OIDC दस्तावेज़ों में दावा किया गया) में सहजता से फिट हो सके।
- **Viper (कॉन्फिगरेशन), Logrus (लॉगिंग), Brotli/compress (संपीड़न)** — परिचालन संबंधी आधारभूत ढाँचा: लचीला कॉन्फिगरेशन, संरचित लॉग, और पेलोड संपीड़न।
- **Angular** — वेब सिंगल-पेज एप्लिकेशन के लिए चुना गया, जो वेरिफिकेशन और मॉनिटरिंग का दृश्यात्मक प्रवेश द्वार है।
- **Python + JavaScript SDKs** — क्लाइंट टीमों को प्रथम श्रेणी की पहुँच प्रदान करने के लिए चुना गया, जिसे OpenAPI/Swagger के माध्यम से दस्तावेज़ीकृत किया गया है।
- **Docker, Kubernetes, Helm** — उत्पादन परिनियोजन के लिए चुना गया जिसमें स्वास्थ्य मॉनिटरिंग और ऑटोस्केलिंग शामिल है, ताकि वेरिफिकेशन फ़्लीट किसी भी आधुनिक सेवा की तरह स्केल कर सके।
- **Prometheus + Grafana** — मेट्रिक्स और डैशबोर्ड के लिए चुना गया, जिससे प्लेटफॉर्म का स्वयं का स्वास्थ्य उतना ही अवलोकन योग्य हो जितना कि उन मॉडलों का जिन्हें यह मॉनिटर करता है।
- **Testify (Go) + node -test/jsdom (वेब)** — **Go** कोर और वेब फ्रंट एंड में परतदार परीक्षण के लिए चुना गया।

स्थिति और ईमानदारी संबंधी टिप्पणियाँ

- **स्थिति:** बीटा। **Go** स्रोत वास्तविक HTTP वेरिफिकेशन लागू करता है (एक पुराना दस्तावेज़ जो वेरिफिकेशन को केवल कॉन्फिगरेशन-आधारित बताता है, वह आकांक्षात्मक और पुराना है — कोड ही प्रामाणिक है)।
- **लाइसेंस:** निर्धारित होना शेष। README में MIT लिखा है जबकि एक Dockerfile लेबल में Apache-2.0 — प्रकाशन से पहले इसे सुलझा लें।
- **प्रदाताओं की संख्या:** README में “12 एडेप्टर” लिखा है लेकिन प्रदाता डायरेक्टरी में लगभग 26 सूचीबद्ध हैं — इसे “12+ / प्रगति में अधिक” के रूप में लें। कई

आकांक्षात्मक “FINAL/COMPLETE” स्थिति फ़ाइलें मौजूद हैं; कोड, दस्तावेज़, और go.mod ही प्रामाणिक हैं।

- रिपॉजिटरी vasic-digital संगठन में है, लेकिन कार्यात्मक रूप से यह **Helix LLM**-इन्फ्रास्ट्रक्चर क्लस्टर की विश्वसनीयता परत है।

प्राथमिकता स्तर: Helix-प्राथमिक (LLM-इन्फ्रास्ट्रक्चर क्लस्टर; LLM/प्रदाता/वेरिफिकेशन मेटाडेटा के लिए एकल स्रोत)। HelixTrack के बाद आता है।